

• Sec - A

Q 1. Find the time complexity of the following code

```
for(int i = 1; i <= 5; i++)  
{  
    Print f("hello");  
}  
for(int i = 5; i > 0; i--)  
{  
    Print f("ju");  
}
```

Ans Each iteration of the first loop takes constant, $O(1)$, to print the string "hello". The loop iterates 5 times, so the total complexity of the first loop is $O(5)$.

Each iteration of the second loop also takes constant time, $O(1)$, to print string "ju". The loop iterates 5 times, so the total time complexity of the second loop is $O(5)$.

Therefore,

total time complexity of the code is $O(5) + O(5) = O(10)$.

Q 2. What is space complexity? Why should we care about space complexity?

Ans Space complexity is the amount space required by an algorithm or program to execute. It is a measure of the resources required to solve a given problem.

We should care about space complexity because it can have a significant impact on the performance of an algorithm or program. If an algorithm requires too much memory, it may not be able to run on a computer with limited memory. Additionally

algorithms with high space complexity can be slower than algorithms with lower space complexity.

Q3. Explain Lower Bound in sorting.

Ans. The lower Bound of a sorting algorithm is the minimum amount of time required to sort a given input size, regardless of the algorithm used. The lower bound of sorting is $O(n \log n)$, which is the time complexity of the merge sort and quicksort algorithm.

To understand why the lower bound of sorting is $O(n \log n)$, we can consider the following: Any sorting algorithm must compare at least two elements in order to sort them. In the worst case, the algorithm will need to compare every pair of elements in the input. This means that the number of comparisons required to sort a given input size is at least $O(n^2)$.

Q4. How are the elements of a 2D array stored in the memory? Explain with example.

Ans. In memory, elements of a 2D array are stored sequentially, row by row using a process called row major order or row-wise storage. Each element in the array is assigned a memory address and the addresses are arranged in a linear manner, with consecutive elements in the same row stored next to each other.

Ex: Consider 2×3 2D array containing integers

```
int arr[2][3] = {
    { 1, 2, 3 },
    { 4, 5, 6 }
};
```

in memory, it should be stored like this:

memory address	Array element	value
0x1000	arr[0][0]	1
0x1004	arr[0][1]	2
0x1008	arr[0][2]	3
0x100c	arr[1][0]	4
0x1010	arr[1][1]	5
0x1014	arr[1][2]	6

Q5. Given the Base address of an array B[1300.... 1900] as 1020 and the size of each element is 2 bytes in the memory. Find the address of B[1700].

Ans

Given:

Base address B = 1020

Lower limit / Lower Bound of subscript LB = 1300

Storage size of one element store in any array

W = 2 Byte.

Subset of element whose address to be found

I = 1700

Formula:

$$\begin{aligned} \text{address of } A[I] &= B + W \times (I - LB) \\ &= 1020 + 2 \times (1700 - 1300) \\ &= 1820 \end{aligned}$$

= Ans

• Sec B: (3x7=21)

Q1. Write an efficient program for printing K largest element in an array. Elements in an array can be in any order.

Examples:

input: [1, 23, 12, 9, 30, 2, 50], k = 3

output: 50, 30, 23

Ans.

```

import java.util.Arrays;
public class KlargestElements
{
    public static void printKlargestElements (int[] arr,
                                              int n, int k)
    {
        Array.sort(arr, (a, b) -> Integer.compare(b, a));
        for (int i = 0; i < k; i++)
        {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }

    public static void main (String[] args) {
        int[] arr = { 1, 23, 12, 9, 30, 2, 50 };
        int n = arr.length;
        int k = 3;

        printKlargestElements(arr, n, k);
    }
}

```

Output: 50, 30, 23

Q2.
Ans.

Explain the operation on data structure in detail.
 operation on data structure refers to the fundamental actions that can be performed on data stored within a specific data structure. Data structures are essential in computer science and programming because they allow you to organize and manage data efficiently.

There are different types of operations that can be performed for the manipulation of data.

Jan 2, 2024 17:31

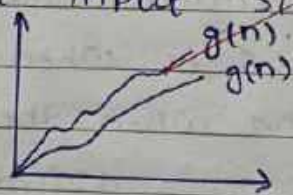
every data structure. Some operations are :-

- (i) Searching :- Searching means to find a particular element in the given data structure. It is considered successful when the required element is found. Searching is the operation which we can perform on data structure like array, linked list, tree, graph, etc.
- (ii) Sorting :- Sorting is a data structure operation that arranges the elements of a data structure in a specific order, such as ascending or descending order. Sorting is a very common operation in computer science, and it is used in a wide variety of applications, such as searching database management and data visualization.
The best sorting algorithm to use depends on the specific data structure being sorted and the size of the data structure.
- (iii) Insertion :- Insertion means to add an element in the given data structure. The operation of insertion is successful when the required element is added to the required data structure. It is unsuccessful in some cases when the size of data structure is full and when there is no space in data structure to add any additional element.
- (iv) Deletion :- Deletion means to delete an element in the given data structure.
- (v) Update :- It updates the data in data structure.
- (vi) Merge :- Merging data of two different orders in a specific order may ascend or descend.
- (vii) Split data :- Dividing data into different sub-parts to make the process complete in less time.

Q3. What are Asymptotic Notations? Explain in detail.

Ans Asymptotic Notations are the mathematical notations used to describe the running time of an algorithm when the input tends towards a particular value or a limiting value. Asymptotic Notation allow to analyze an algorithm's running time by identifying its behaviour as its input size grows. It is also referred to as an algorithm's growth rate. There are mainly three asymptotic notations :-

(i) Big-O Notation (O-Notation) :- It represents the upper bound of the running time of an algorithm. Therefore it gives the worst-case complexity of an algorithm. It is widely used notation for asymptotic analysis. It returns the highest possible output value (big-O) for a given input. For ex:- If an algorithm has a time complexity of $O(n)$, it means algorithm run time grows linearly with the input size n .

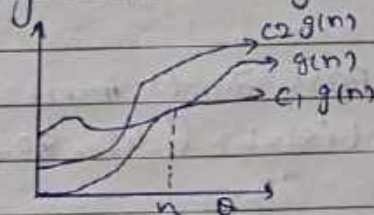


(ii) Omega Notation (Ω -Notation) :- It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.

Omega Notation represents the lower bound on the running time of an algorithm thus it provides the best case time complexity.



(iii) Theta Notation (Θ -Notation) :- it encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm. it is used for analyzing the average-case complexity of an algorithm.



$\Theta(g(n)) = \{ f(n) : \text{there exist +ve constant } c_1, c_2 \text{ \& } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}$.

• Section - C (3x11=33)

Q1. Implement quick sort algorithm on the below list :
97, 82, 450, 99, 45, 99, 101, 203, 23. write down all the implementation passes and the changed list, also let us know for what kind of input it will generate worst case time complexity.

Ans: Applying insertion sort algorithm to the given list
[97, 82, 450, 99, 45, 99, 101, 203, 23]

Pass 1 (initial list)

List : [97, 82, 450, 99, 45, 99, 101, 203, 23]

Pass 2. (insert 82 into the sorted part)

List : [82, 97, 450, 99, 45, 99, 101, 203, 23]

Pass 3 (insert 450 into the sorted part)

List : [82, 97, 450, 99, 45, 99, 101, 203, 23]

Pass 4 (insert 99 into sorted part)

List: [82, 97, 99, 450, 45, 99, 101, 230, 23]

Pass 5 (insert 45 into sorted part)

List: [45, 82, 97, 450, 99, 101, 230, 23]

Pass 6 (insert 99 into sorted part)

List: [45, 82, 97, 99, 99, 450, 101, 230, 23]

Pass 7 (insert 101 into sorted part)

List: [45, 82, 97, 99, 99, 101, 450, 230, 23]

Pass 8 (insert 230 into sorted part)

List: [45, 82, 97, 99, 99, 101, 230, 450, 23]

Pass 9 (insert 23 into sorted part)

List: [23, 45, 82, 97, 99, 99, 101, 230, 450]

After all the passes, the list is fully sorted in ascending order.

Insertion sort is an $O(n^2)$ algorithm in the worst case. It has its worst case time complexity when the input is in reverse order, meaning that you have to insert each element at the beginning of the sorted portion, leading to many shifts.

Q2. Given an integer N and a list arr. sort the array using the bubble sort algorithm.

Example 1:

Input:

$N = 5$

arr [] = { 4, 1, 3, 9, 7 }

Output: 1, 3 4 7 9

Ans.

```

public class BubbleSort {
    public static void main (String [] args) {
        int [] arr = { 4, 1, 3, 9, 7 };
        bubbleSort (arr);
        System.out.println(" Sorted array");
        for (int i=0; i< arr.length; i++)
        {
            System.out.print(arr[i] + " ");
        }
    }

    public static void bubbleSort (int [] arr) {
        for (int i=0; i< arr.length-1; i++)
        {
            for (int j=0; j< arr.length-i-1; j++)
            {
                if (arr[j] > arr[j+1])
                {
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
    }
}

```

output: Sorted array:
1 3 4 7 9

Q3. Write a linear search algorithm to find the element in an array. Also analyze its behaviour in the worst, best and average cases.

Ans Linear search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.

Implementation of linear search algorithm.

```

↳ public class LinearSearch {
    public static int linearSearch(int[] arr, int target)
    {
        for (int i = 0; i < arr.length; i++)
        {
            if (arr[i] == target)
            {
                return i;
            }
        }
        return -1;
    }
}

```

```

public static void main (String[] args)
{
    int[] arr = { 4, 1, 3, 9, 7 };
    int target = 9;

```

```

    int index = linearSearch(arr, target);
    if (index != -1)

```

```

    {
        System.out.println("The target element is found at index: " + index);
    }
}

```

```

    }
    else
    {
        cout << "The target element is not found in the array.";
    }
}

```

Output:- The target element is found at index: 3

- Behaviour of Linear Search algorithm in terms of worst, best and average cases:

- Worst Case:- The worst case scenario occurs when the target element is either the last element in the array or not present in the array. In this case, the algorithm must traverse the entire array before determining that the element is not there. The worst case time complexity is $O(n)$, where n is the size of the array.
- Best case:- The best case scenario occurs when the target element is the first element in the array. In this case, the algorithm finds the elements in the first comparison without iterating further. The best case time complexity is $O(1)$.
- Average Case:- In the average case scenario, the target element can be anywhere in the array with equal probability. On average, the linear search algorithm needs to examine half of the elements in the array before finding the target. Thus, the average time complexity is $O(n/2)$ which simplifies to $O(n)$ in Big-O-Notation.

← The end

• Sec-A:
Q1. Calculate the complexity for the following code:

```
for (i = 1; i <= n2; i++)  
{  
    printf("hello World");  
}
```

Ans The loop runs from " $i = 1$ " to " $i = n2$ " which means it will execute $n2$ times. In terms of time complexity analysis, this would be denoted as $O(n2)$.
So,

Time complexity of this code is $O(n2)$, where $n2$ is the value of loop's upper bound.

Q2. Describe the types of Data Structures.

Ans Data Structures are a way of organizing data in a computer so that it can be used efficiently. They provide a way to store, retrieve and manage data in a variety of ways.

• There are mainly two types of data structures.

(i) Primitive D.S.:- The primitive D.S. are primitive data types. The int, char, float, double and pointer are the primitive data structures that can hold the single values.

(ii) Non-Primitive D.S.:- The non-primitive data structure is divided into two types.

(a) Linear Data Structure:- A data structure called linear if all of its elements are arranged in the linear order. ex = Array, linked list, stack, queue, etc.

(b) Non-Linear Data Structure:- The D.S. does not form a sequence way. i.e. each item or element is connected with two or more items in non-linear arrangements. ex = Tree, graph, hash table, etc.

Q3. Explain Abstract Data type with example.

Ans An abstract data type (ADT) is a high-level data structure that is defined by its behavior and the operations that can be performed on it, rather than by its implementation details. This allows different implementations of the same ADT to be used without changing the code that uses the ADT.

Examples of some ADTs

Stack, Queue, Map, Graph, Tree, List, etc.

Q4. What is time complexity? Why should we care about time complexity?

Ans Time complexity is a measure of how long an algorithm takes to run as a function of the size of the input. It is an important concept in computer science because it allows us to compare the efficiency of different algorithms and to choose the most efficient algorithm for a particular problem.

We should care about time complexity because it can have a significant impact on the performance of our software applications. By choosing efficient algorithms, we can make our applications faster and more responsive.

Here are some tips for choosing efficient algorithms:

- Use the simplest algorithm that gets the job done.
- Be aware of the time complexity of the algorithm that you are using.
- Use a profiler to identify the bottlenecks in your code.

Q5. What is the difference betⁿ. File structure and Storage structure? Explain in detail.

Ans →

Ans File structure is the logical organization of data within a file. It defines how the data is stored in the file, such as the order of the data, the format of the data. File structure is independent of the physical storage medium.

Storage structure is the physical organization of data on a storage medium. It defines how the data is actually stored on the storage medium, such as the physical layout of the data, the addressing scheme used to access the data. Storage structure is dependent on the physical characteristics of the storage medium.

Here is a table that summarizes the key differences betⁿ file structure and storage structure.

Characteristic	File Structure	Storage Structure
(i) Definition	Logical organization of data within a file.	Physical organization of data on a storage medium.
(ii) Dependence on	Independent of the storage medium.	Dependent on the storage medium.
(iii) Example	File system	Disk layout.

• see - B ($3 \times 7 = 21$)

Q 2 (a) How are the elements of a 1D array stored in the memory? Explain with example.

(b) An array X $[-15 \dots 10, 15 \dots 40]$ requires one byte of storage. If the beginning location is 1500, determine the location of $X[15][20]$.

Ans $\rightarrow$ (a). The element of 1D array are stored in memory

Jan 2, 2024, 17:32

in a contiguous block, with each element taking up the same amount of space. The array base address is the address of the first element in the array. Each element is stored at a memory address that is equal to the array base address plus the size of the element type times the element index.

For ex:- 1D array elements are

array = {1, 2, 3, 4, 5}

If the array base address is 1000, then the elements of the array will be stored in memory as follows:

Address	Value
1000	[1]
1004	[2]
1008	[3]
1012	[4]
1016	[5]

Ans:- The array X has a size of (-15, 40), which means that it has 55 columns and 56 rows. Each element of the array requires one byte of storage.

Therefore,

the location of $X[15][20]$ can be calculated as follows.

$$\text{location} = \text{beginning-location} + (\text{row-index} \times \text{no. of columns} + \text{column index})$$

$$\begin{aligned} \Rightarrow \text{location} &= 1500 + (15 \times 55 + 20) \\ &= 2345 \end{aligned}$$

Therefore, the location of $X[15][20]$ is 2345.

(P.T.O.)

Q. 2 (b) ⇒ Beginning Location : 1500
 offset for Row : 15 bytes (15 rows * 1 byte/row)
 offset for column : 20 bytes (20 column * 1 byte/column)

Location of $X[15][20] = \text{Beginning Location} + \text{offset for Row} + \text{offset for column}$

$$\text{Location of } X[15][20] = 1500 + 15 + 20 \\ = 1535$$

So, the location of $X[15][20]$ in memory is 1535.

Q. 2. Which algorithm is used for sorting algorithm that does not use extra space for manipulating the input? Also, write an algorithm and find the best case time efficiency of the sort.

Ans. The algorithm we are looking for is the "In Place Sorting" Algorithm. It sorts the elements of an array or list by rearranging the elements within an array itself without using additional memory for copying or moving elements to another data structure.

Common in place sorting algorithm is QuickSort. It is a divide and conquer based algorithm that works by selecting a "Pivot" element & partitioning the other element into sub arrays.

Algorithm of Quick sort (working) -

- Algo → QuickSort (In-Place)
- Input → An array A, start index-low, end-index-high
- Output → Sorted array A.

(i) If $low < high$:

(a) Partition the array A & obtain a Pivot Index.

(b) Recursively call Quicksort on sub-Array from low to Pivot index.

(ii) Partitioning :-

(a) Choose a Pivot index element.

(b) initialize two pointers : $i = low - 1$ & $j = low$

(c) loop over element from low to $high - 1$.

(d) swap $A[i+j]$ & $A[high]$ to place the Pivot in its correct position.

(e) Return $i+1$ as the Pivot index.

(iii) The array A is now sorted.

~~Now,~~

The best-case time efficiency of the Selection Sort algorithm is $O(n^2)$, which is the same as the worst-case time efficiency. This is because the Selection Sort algorithm always needs to find the smallest element in the unsorted portion of the array, which takes $O(n)$ time. In the best case, the input array is already sorted, so the selection sort algorithm will simply need to copy the array to the output array, which takes $O(n)$ time.

Q3. Analysis of the time complexity of Bubble sort, selection sort and insertion sort methods with example

Ans. Bubble Sort :- It is the simple comparison-based Sorting Algorithm that repeatedly steps through the list, compares the elements and then swaps them if they are in wrong order.

Time complexity :-

(i) Best case - $O(n)$ → occurs when array is fully sorted.

(ii) Worst case - $O(n^2)$ → occurs when array is sorted in Reverse.

(iii) Average case - $O(n^2)$ → occurs when array is partially sorted.

* Selection Sort :- it is another simple sorting algorithm that divides the input list into two parts. It repeatedly selects the smallest element from unsorted array.

Time complexity :-

(i) Best case - $O(n^2)$ → Occurs when array is sorted in Reverse.

(ii) Worst case - $O(n^2)$ → performs same no. of comparisons.

(iii) Average case - $O(n^2)$ → Occurs when array is partially sorted.

* Insertion Sort :- it is also a simple sorting algorithm that builds the final sorted array one item at a time.

Time complexity :-

(i) Best case: $O(n)$ → occurs when array is already sorted.

(ii) Worst case: $O(n^2)$ → occurs when array is sorted in Reverse.

(iii) Average case: $O(n^2)$ → in most case, average is Quadratic also.

• See - C (3x11=33)

Q(1) Sort the elements 54, 26, 93, 17, 77, 31, 44, 55 & 20 using insertion sort algorithm.

Ans. To sort the elements 54, 26, 93, 17, 77, 31, 44, 55, 20 using insertion sort algo, that is given below -

```
Public class InsertionSort {
    Public static void main (String [] args) {
        Int [] array = { 54, 26, 93, 17, 77, 31, 44, 55, 20 };
        InsertionSort (array);
```

```
    for (int i=0; i < array.length; i++)
    {
        System.out.println (array[i] + " ");
    }
}
```

```
Public static void InsertionSort (int [] array)
{
    for (int i=1; i < array.length; i++)
    {
```

```
        int key = array[i];
        int j = i-1;
        while (j >= 0 && array[j] > key)
        {
```

```
            array[j+1] = array[j];
            j--;
        }
```

```
        array[j+1] = key;
    }
```

```
}
```

Output: 17 20 26 31 44 54 55 77 93

Q2

Write an algorithm of selection sort. Also sort the given array: -6, 9, 2, 8, 5, 4, 10, 1, 3.

Ans

The selection sort algorithm works by repeatedly finding the smallest (or largest) element in the unsorted portion of the array and swapping it with the element at the beginning of the unsorted portion. This process is repeated until the entire array is sorted.

Here are pseudocode of selection sort:-

```

Function selection
→ P.S.V. selectionSort (int[] arr)
{
    for (int i = 0; i < arr.length - 1; i++)
    {
        if (arr[j] < arr[smallestIndex])
        {
            smallestIndex = j;
        }
    }
    if (smallestIndex != i)
    {
        int temp = arr[i];
        arr[i] = arr[smallestIndex];
        arr[smallestIndex] = temp;
    }
}
}
}

```

To sort the given array [-6, 9, 2, 8, 5, 4, 10, 1, 3] using the selection sort algorithm, we can

Use the following code :-

```
#
public class SelectionSort {
    public static void main (String [] args)
    {
        int [] array = {-6, 9, 2, 8, 5, 4, 10, 1, 33};
        selectionSort (array);

        for (int i = 0; i < array.length; i++)
        {
            System.out.print (array [i] + " ");
        }
    }

    public static void selectionSort (int [] array)
    {
        for (int i = 0; i < array.length - 1; i++)
        {
            int smallestIndex = i;

            for (int j = i + 1; j < array.length; j++) {
                if (array [j] < array [smallestIndex])
                {
                    smallestIndex = j;
                }
            }

            if (smallestIndex != i) {
                int temp = array [i];
                array [i] = array [smallestIndex];
                array [smallestIndex] = temp;
            }
        }
    }
}
```

Output :-

-6 1 2 3 4 5 8 9 10

93. Write a binary search algorithm to find the element in an array with a suitable example. Also analyze its behaviour in worst, best and average case.

Ans. Binary Search is a search algorithm that finds the position of a target value within a sorted array. It works by repeatedly comparing the target value to the middle element of the array. If the target value is equal to the middle element, the algorithm returns the middle index. If the target value is less than the middle element, the algorithm searches the lower half of the array. If the target value is greater than the middle element, the algorithm searches the upper half of the array. This process is repeated until the target value is found or the algorithm determines that the target value is not in the array.

example of Binary search Algorithm code: —

```
↳ public class BinarySearch {  
    public static int binarySearch(int[] arr, int target)  
    {  
        int low = 0;  
        int high = arr.length - 1;  
  
        while (low <= high) {  
            int mid = (low + high) / 2;  
  
            if (arr[mid] == target) {  
                return mid;  
            }  
  
            else if (arr[mid] < target) {
```

```

        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}
return -1;
}

public static void main (String[] args) {
    int[] arr = { 1, 3, 5, 7, 9 };
    int target = 5;
    int index = binarySearch (arr, target);
    if (index == -1) {
        System.out.println (" The target value is not in the array. ");
    } else {
        S.O.P (" The Target value is at Index " + index + ". ");
    }
}

```

$\xrightarrow{\text{Output:}}$
 The Target value is at index 2.

- Worst case :- $O(\log n) \rightarrow$ occurs when Binary search needs max. no. of comparisons.
- Best case :- $O(1) \rightarrow$ Linear $\rightarrow$ occurs when target element is found in middle of the array in 1st comparison.
- Average case :- $\log_2 n \rightarrow$ occurs when Binary search requires $\log_2 n$ comparisons.

Subby
 16/10/2023

The ans. (22BCON1068)